Programming EEPROM

EEPROM

Amer Iqbal Qureshi | Mikrotronics Pakistan

EEPROM Programming

Permanent Data Storage on AVR Microcontrollers

Learning Objectives

After completing this chapter, the reader will be able to:

- Explain what EEPROM is.
- Differentiate EEPROM, SRAM and Flash memory.
- Understand how EEPROM stores information.
- Access EEPROM through AVR registers.
- Write and read bytes directly from hardware.
- Use the MikroDuino EEPROM library.
- Store variables, structures and configuration data.
- Design applications that preserve EEPROM life.

Introduction

One limitation of SRAM is that everything stored inside it disappears when power is removed. Imagine building a digital thermometer.

The user enters

- preferred temperature unit
- alarm limits
- display brightness

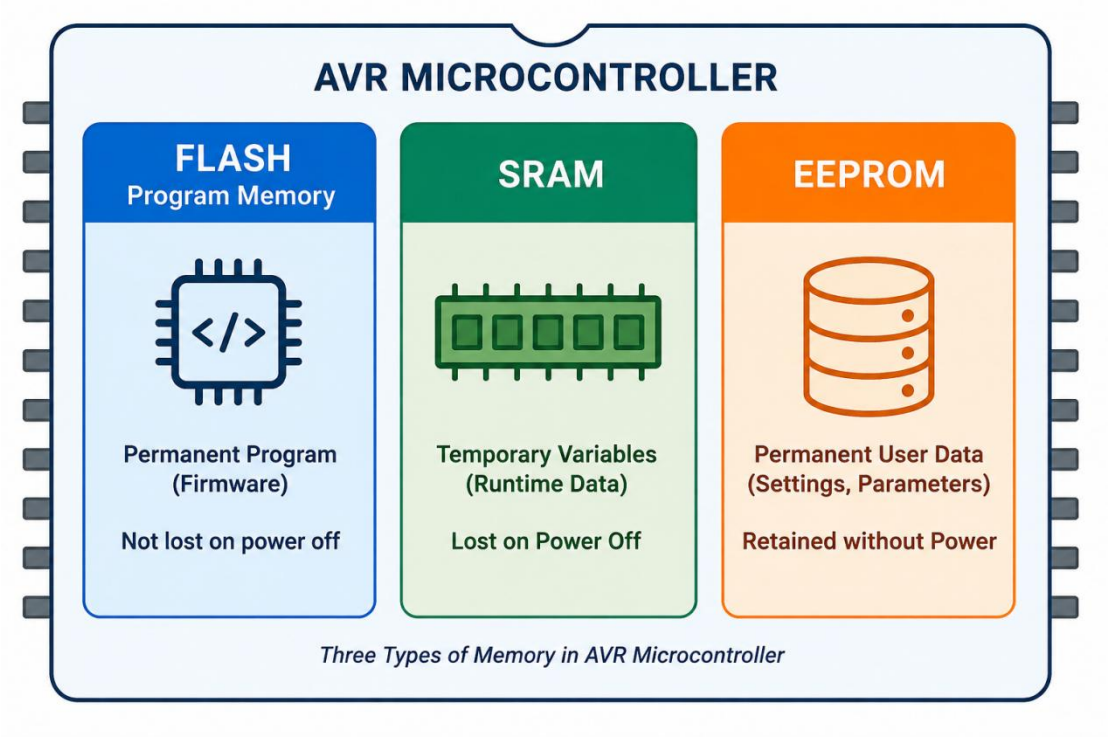
If these values were stored only in SRAM, every setting would disappear whenever the board is switched off. Fortunately, AVR microcontrollers contain another type of memory called **EEPROM**.

EEPROM remembers information even when power is removed.

This makes it ideal for storing:

- Device settings
- User preferences
- Calibration values
- Wi-Fi passwords
- Device serial numbers
- Operating modes
- Counters
- Last saved state

Unlike Flash memory, EEPROM can be modified while the application is running.



What is EEPROM?

EEPROM stands for **Electrically Erasable Programmable Read Only Memory**

Although the name contains "Read Only", modern EEPROM can be rewritten thousands of times. It is called read-only because normal programs primarily read stored information, while writing requires a special hardware sequence.

EEPROM is non-volatile memory. Non-volatile means

the contents remain even after the power supply is disconnected.

This is similar to the SSD inside a computer.

EEPROM vs SRAM vs Flash

Property	SRAM	Flash	EEPROM
Power required to retain data	Yes	No	No
Can store program	No	Yes	No
Can store variables	Yes	Yes	Yes
Fast access	Very Fast	Moderate	Slow
Can change while program runs	Yes	Difficult	Yes
Typical endurance	Unlimited	10,000 cycles	100,000 cycles

Memory Types in AVR Microcontrollers – Comparison

Feature	FLASH	SRAM	EEPROM
	 Program Memory	 Temporary Variables	 Permanent User Data
Power required to retain data	No	Yes	No
Can store program	Yes	No	No
Can store variables/data	Yes (But limited write)	Yes	Yes
Access speed	Moderate (µs range)	Very Fast (ns range)	Slow (ms range)
Can change while program runs	Difficult (Requires erase)	Yes	Yes
Typical endurance / write cycles	~10,000 cycles	Unlimited	~100,000 cycles
Data retention	20+ years	Lost when power off	20+ years
Typical use	Program code, Firmware	Variables, Buffers, Stack	Settings, Calibration, User Preferences

Where EEPROM is Used

- Car radio
- Stores favorite stations.
- Television
- Stores channel list.
- Digital weighing scale
- Stores calibration.
- Blood glucose meter
- Stores previous readings.
- Industrial controller
- Stores machine parameters.
- Robot
- Stores PID constants.
- Microcontroller
- Stores user configuration.

How EEPROM Stores Data

Before we learn how to read and write EEPROM, it is worth asking an interesting question.

How can a tiny memory cell remember information even after the power has been turned off?

After all, when we disconnect power from the microcontroller, everything stops. No electricity is flowing. No clock is running. Yet when power is restored hours, days, or even years later, the data is still there.

A Bucket That Remembers

Imagine that every EEPROM memory cell is like a tiny bucket. This bucket can either be **empty** or **filled with tiny invisible balls**.

Instead of water, however, the bucket stores **electrons**.

There are only two possible conditions.

- **Bucket empty → Logic 1**
- **Bucket contains electrons → Logic 0**

The bucket does not leak. Once electrons are placed inside, they remain trapped until the microcontroller deliberately removes them. This trapped electrical charge is what represents the stored data. Unlike an ordinary electrical signal, which disappears the moment power is removed, the trapped electrons stay exactly where they are.

That is why EEPROM remembers its contents even when the device is completely switched off.

One Bit per Bucket

Each bucket stores only **one bit**. If the bucket is empty, it represents a binary **1**. If the bucket contains trapped electrons, it represents a binary **0**.

Since one byte consists of eight bits, storing a single byte requires **eight separate EEPROM cells**.

For example,

```
Byte: 10100110

Bit 7   Bit 6   Bit 5   Bit 4   Bit 3   Bit 2   Bit 1   Bit 0

 1       0       1       0       0       1       1       0
```

Each of these bits is stored in its own tiny bucket inside the EEPROM array.

The Floating Gate

Of course, the microcontroller does not actually contain buckets.

Instead, each EEPROM cell is built around a special type of transistor called a **floating-gate transistor**. You do not need to understand semiconductor physics to use EEPROM effectively. Simply think of the floating gate as the bucket that traps electrons. When electrons are stored on the floating gate, the transistor behaves differently from when it is empty.

The EEPROM hardware senses this difference and interprets it as either a **0** or a **1**.

Writing Data

Now suppose we want to store the number

```
0x5A
```

The microcontroller must decide which buckets should contain electrons and which should remain empty. To do this, the AVR activates special internal circuitry that produces a voltage much higher than the normal supply voltage.

This high voltage pushes electrons into selected floating gates while leaving others unchanged.

The entire operation takes approximately **3.4 milliseconds**, which explains why writing to EEPROM is much slower than writing to SRAM.

Reading Data

Reading is much easier.

The microcontroller simply checks whether each bucket contains trapped electrons. If electrons are present, the hardware reports a binary **0**. If no electrons are present, it reports a binary **1**.

Since reading does not move any electrons, it causes virtually no wear to the EEPROM and can be performed millions of times.

Erasing Data

Eventually, we may wish to store new information. Some of the buckets that currently contain electrons must first be emptied. The EEPROM hardware again uses its special high-voltage circuitry, but this time it removes the trapped electrons from the floating gate.

Once emptied, the cell returns to its natural state.

```
Empty Bucket  → Logic 1
```

This is why a freshly erased EEPROM location always reads as:

```
0xFF
```

Every bit is a **1**, meaning every bucket is empty.

Why EEPROM Has a Limited Lifetime

Although the bucket analogy is useful, imagine that each bucket is made from a very delicate material. Every time electrons are forced into the bucket and later removed, the bucket experiences a tiny amount of wear. One write operation causes almost no damage.

Even a thousand writes make little difference.

However, after roughly **100,000 write cycles**, some buckets may no longer hold electrons reliably. For this reason, EEPROM is intended for storing values that change **occasionally**, not continuously. Configuration settings, calibration constants, operating modes, and serial numbers are excellent candidates because they are written infrequently but read often.

7. EEPROM Registers

In the previous section, we learned that EEPROM stores information as tiny electrical charges trapped inside floating-gate transistors. However, the CPU cannot access these memory cells directly. Instead, it communicates with the EEPROM hardware through a small group of special-purpose registers.

Think of these registers as a receptionist sitting between the CPU and the EEPROM memory.

Whenever the processor wants to read or write a byte, it first tells the receptionist:

- **Which memory location to access**
- **What data should be written**
- **Whether the operation is a read or a write**

These tasks are performed by three dedicated registers:

- **EEAR** – EEPROM Address Register
- **EEDR** – EEPROM Data Register
- **EECR** – EEPROM Control Register

Together, these registers provide complete control over the EEPROM hardware.

EEPROM Address Register (EEAR)

The first question the EEPROM controller asks is,

> **"Which EEPROM memory location do you want to access?"**

The answer is stored in the **EEPROM Address Register (EEAR).**

Whenever the CPU wants to read or write EEPROM, it first loads the desired memory address into EEAR. For example, suppose we want to access EEPROM location number **120**.

The processor simply writes the value **120** into the EEAR register. The EEPROM controller now knows exactly which memory cell is being selected. You can think of EEAR as writing the **house number** before sending a letter. Without the address, the EEPROM controller would not know where to store or retrieve the data.

Reading EEPROM Using Registers

Reading data from EEPROM is one of the simplest operations performed by the AVR microcontroller. Unlike writing, no erase cycle or special timing sequence is required. The EEPROM controller simply retrieves the contents of the requested memory location and makes it available to the CPU.

Think of EEPROM as a large filing cabinet.

Suppose you ask someone,

> **"Please bring me the document stored in drawer number 16."**

The person first goes to drawer **16**, opens it, retrieves the document, and hands it to you.

The AVR follows exactly the same procedure.

- Tell the EEPROM which memory location you want.
- Tell it to perform a read operation.
- Collect the data that it returns.

This process always follows the same sequence.

Step 1 — Select the EEPROM Address

The first step is to tell the EEPROM controller which memory location should be accessed. This is done by loading the required address into the **EEPROM Address Register (EEAR).**

Suppose we wish to read the byte stored at EEPROM address **0x10** (decimal 16).

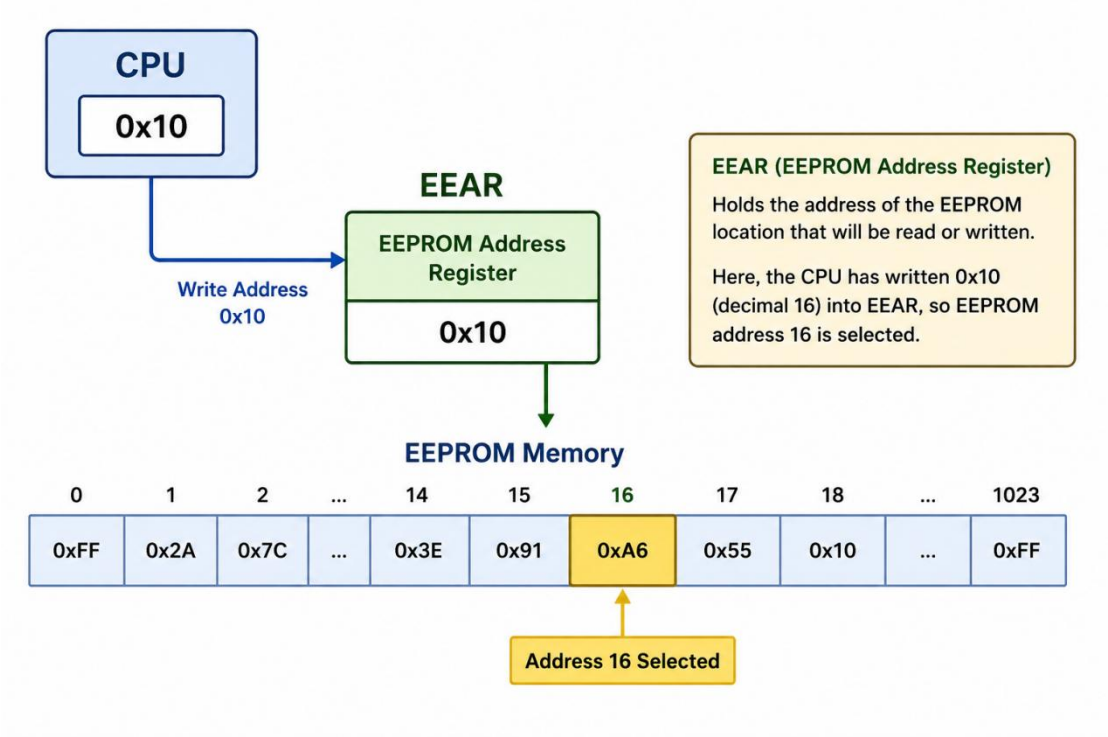
```
EEAR = 0x10;
```

Here,

- **EEAR** is the EEPROM Address Register.
- **0x10** is the hexadecimal address of the required EEPROM location.

After this instruction, the EEPROM controller knows exactly which byte should be read.

At this stage, however, **no data has been transferred yet**. We have only selected the memory location.



Step 2 — Start the Read Operation

Once the address has been selected, the processor tells the EEPROM controller to begin reading. This is done by setting the **EERE (EEPROM Read Enable)** bit inside the **EEPROM Control Register (EECR).**

```
EECR |= (1 << EERE);
```

This will set the EERE bit of the EECR register to 1.

What Happens Internally?

The moment the EERE bit is set,

the EEPROM controller immediately

- goes to the selected address,
- reads the stored byte,
- copies that byte into the EEDR register, and
- automatically clears the EERE bit.

This entire process requires only **one CPU clock cycle**. The programmer does not have to wait.

Step 3 — Read the Data

Once the EEPROM controller has copied the data into **EEDR**, the processor simply reads that register.

```
uint8_t value = EEDR;
```

This line creates a variable called **value** and copies the contents of EEDR into it.

Complete Register-Level Example

The complete EEPROM read operation requires only three instructions.

```
// Select EEPROM address 0x10EEAR = 0x10;
// Start EEPROM readEECR |= (1 << EERE);
// Read the byte from EEDRuint8_t value = EEDR;
```

How the Three Registers Work Together

Notice how each register performs a different job.

Register	Purpose
EEAR	Selects the EEPROM memory address.
EECR	Starts the read operation by setting the EERE bit.
EEDR	Holds the byte that has been read from EEPROM.

Each register has a specific responsibility.

The processor always follows the same sequence:

1. **Choose the address** using **EEAR**.
2. **Request the read** using **EECR**.
3. **Collect the data** from **EEDR**.

Writing EEPROM Using Registers

Reading data from EEPROM is quick and straightforward. The EEPROM controller simply retrieves the stored byte and places it into the **EEDR** register for the CPU to use.

Writing data, however, is a much more careful process.

Unlike SRAM, where data can be changed almost instantly, EEPROM must physically alter the electrical charge stored inside its floating-gate transistors. This process takes time and must be performed with great care to avoid accidentally corrupting important data.

For this reason, writing to EEPROM is significantly slower than reading from it.

Why Is Writing Slower?

Imagine writing on a whiteboard.

Reading what is already written takes only a second.

However, changing the text requires several steps:

- ✓ Find the correct location.
- ✓ Erase the old writing.
- ✓ Write the new text.
- ✓ Check that everything has been written correctly.

EEPROM performs a very similar sequence internally.

When a write operation begins, the EEPROM hardware must:

1. Locate the selected memory cell.
2. Erase the previous electrical charge.
3. Program the new charge onto the floating gate.
4. Verify that the new value has been stored correctly.

All of these operations require approximately **3.4 milliseconds** on most AVR microcontrollers. Although this may seem very fast to us, it is extremely slow compared to the processor, which can execute tens of thousands of instructions during the same time.

The EEPROM Write Sequence

Writing a byte into EEPROM always follows the same sequence.

1. Wait until any previous write operation finishes.
2. Select the EEPROM address.
3. Load the data to be written.
4. Enable the write operation.
5. Start the programming cycle.

The AVR hardware enforces this sequence to protect the EEPROM from accidental writes.

Step 1 — Wait Until EEPROM Is Ready

Before starting a new write operation, the processor must ensure that the EEPROM is not already busy writing another byte.

This is done by checking the **EEPE (EEPROM Programming Enable)** bit.

```
while (EECR & (1 << EEPE));
```

This statement repeatedly checks the EEPE bit.

As long as EEPE remains **1**, the EEPROM hardware is still busy.

The processor waits here until EEPE becomes **0**, indicating that the previous write operation has completed.

Think of this as waiting for a washing machine to finish before opening the door.

Step 2 — Select the EEPROM Address

Next, choose the memory location that will receive the new data.

```
EEAR = 0x20;
```

This loads hexadecimal address **0x20** (decimal 32) into the EEPROM Address Register.

The EEPROM controller now knows where the new byte should be stored.

Step 3 — Load the Data

Now place the byte that should be written into the EEPROM Data Register.

```
EEDR = 0x55;
```

At this point, nothing has been written yet.

The data is simply waiting inside the EEDR register until the write operation begins.

Step 4 — Enable Writing (EEMPE)

Before the EEPROM can be programmed, the AVR requires an additional safety step.

```
EECR |= (1 << EEMPE);
```

This sets the **EEPROM Master Programming Enable (EEMPE)** bit.

Think of EEMPE as the **safety cover on a missile launch button**.

Lifting the cover does **not** launch the missile.

It merely allows the launch button to be pressed. Similarly, setting EEMPE does **not** write anything.

It simply tells the EEPROM controller,

> **"I really intend to perform a write operation."**

The Four-Cycle Rule

One of the most interesting safety features of AVR EEPROM is that the EEMPE bit remains active for only **four CPU clock cycles**. This means the processor has only a tiny amount of time to begin the write operation.

If the write command is not issued within those four clock cycles, the EEMPE bit automatically clears itself, and the entire sequence must be repeated.

This mechanism prevents accidental EEPROM writes caused by software bugs or unexpected electrical disturbances. Think of it as a security door that unlocks for only a fraction of a second before automatically locking again.

Step 5 — Start Programming (EEPE)

Once EEMPE has been enabled, the processor immediately starts the write operation.

```
EECR |= (1 << EEPE);
```

Setting the **EEPE** bit tells the EEPROM controller,

> **"Begin programming now."**

The EEPROM hardware now works independently of the CPU.

Internally it

1. erases the previous contents,
2. writes the new byte,
3. verifies the stored value,
4. clears EEPE when finished.

During this time, the CPU may continue executing other instructions, although another EEPROM write cannot begin until EEPE returns to zero.

Complete Register-Level Example

The complete sequence for writing **0x55** into EEPROM address **0x20** is shown below.

```
// Wait until any previous write finisheswhile (EECR & (1
<< EEPE));
```

```
// Select EEPROM addressEEAR = 0x20;
// Load the data byteEEDR = 0x55;
// Enable EEPROM programmingEECR |= (1 << EEMPE);
// Start the write operationEECR |= (1 << EEPE);
```

Understanding Each Line

Statement	What It Does
`while (EECR & (1<<EEPE));`	Waits until the EEPROM is no longer busy.
`EEAR = 0x20;`	Selects EEPROM address **0x20** (decimal 32).
`EEDR = 0x55;`	Places the data byte **0x55** into the EEPROM Data Register.
`` `EECR ``	`= (1<<EEMPE);` `
`` `EECR ``	`= (1<<EEPE);` `

Putting It All Together

Writing to EEPROM involves three registers working together, just as they did during a read operation, but with a few additional safety steps.

- **EEAR** tells the EEPROM **where** to write.
- **EEDR** tells it **what** to write.
- **EECR** tells it **when** to start writing.

Because writing permanently changes non-volatile memory, the AVR requires the programmer to deliberately unlock the write mechanism using **EEMPE** before setting **EEPE**. This two-step process, together with the **four-clock-cycle rule**, ensures that EEPROM contents cannot be modified accidentally, making stored configuration data both reliable and secure.

Introducing the MikroDuino EEPROM Library

In the previous sections, we learned how to access the EEPROM by directly manipulating the AVR hardware registers. Although this approach gives us complete control over the EEPROM hardware, it also requires us to remember a specific sequence of operations every time we want to read or write a single byte.

For every write operation, we must:

- Check whether the EEPROM is busy.

- Load the memory address into EEAR.
- Load the data into EEDR.
- Enable EEMPE.
- Set EEPE within four clock cycles.

Similarly, every read operation requires selecting the address, setting the **EERE** bit, and retrieving the data from **EEDR**.

While these operations are not difficult, writing the same register-level code repeatedly quickly becomes tedious and makes programs longer and harder to understand.

Fortunately, the **MikroDuino SDK** provides a much simpler solution.

Why Use a Library?

One of the main goals of the MikroDuino SDK is to allow programmers to focus on solving problems rather than worrying about the details of the underlying hardware.

Instead of manually manipulating EEPROM registers every time, the SDK provides a high-level EEPROM library that performs all the low-level operations automatically.

Whether you are reading a single byte or storing an entire structure, the library internally executes the correct register sequence, including all necessary safety checks.

This results in programs that are:

- Easier to read
- Easier to write
- Less prone to programming errors
- More portable across supported AVR devices

For beginners, this means learning EEPROM programming becomes much less intimidating. For experienced programmers, it means writing cleaner and more maintainable code.

Including the EEPROM Library

Before using any EEPROM functions, the appropriate header file must be included in your program.

```
#include <mikroduino/eeprom.hpp>
```

This header file contains the declarations of all EEPROM-related classes and functions provided by the MikroDuino SDK. Without including this file, the compiler would not recognize the EEPROM library.

Using the MikroDuino Namespace

Like the other libraries in the MikroDuino SDK, the EEPROM library is contained inside the **MikroDuino** namespace. To avoid writing the namespace repeatedly, most programs include the following statement:

```
using namespace MikroDuino;
```

This allows the programmer to access the EEPROM object directly.

Instead of writing

```
MikroDuino::EEPROM.read(10);
```

we can simply write

```
EEPROM.read(10);
```

This makes the program shorter and easier to read.

The EEPROM Object

The MikroDuino SDK provides a ready-to-use global object named **EEPROM**. You do not need to create an instance of this class yourself. Simply include the library, and the object is immediately available for use.

```
EEPROM
```

Think of the EEPROM object as a **smart assistant** that knows how to communicate with the EEPROM hardware.

Whenever you ask it to read or write data, it automatically performs all the register-level operations that we learned in the previous sections.

For example, when you call

```
EEPROM.write(100, 0x55);
```

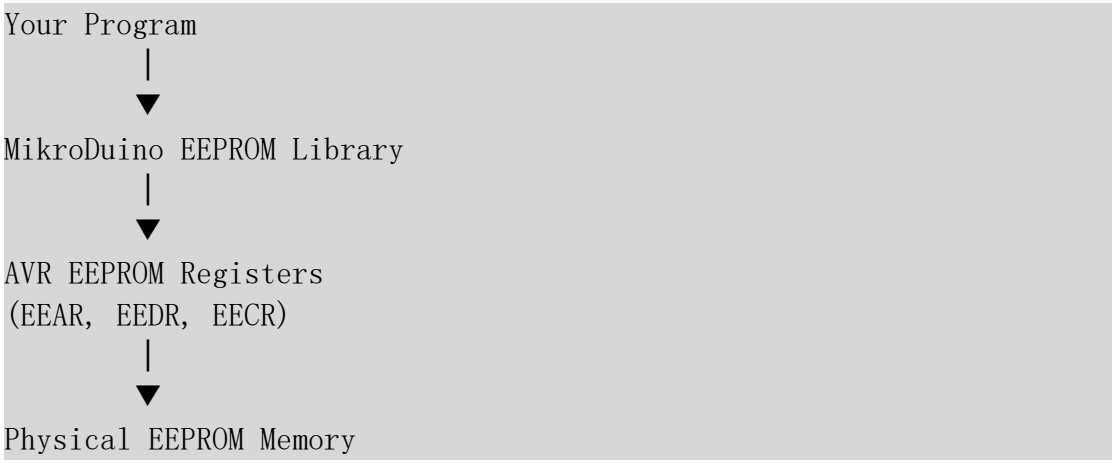
the library internally performs operations similar to:

- Wait until the EEPROM is ready.
- Load address 100 into EEAR.
- Load 0x55 into EEDR.
- Set EEMPE.
- Set EEPE.
- Wait for the write operation to complete if necessary.

All of these hardware details remain hidden from the programmer.

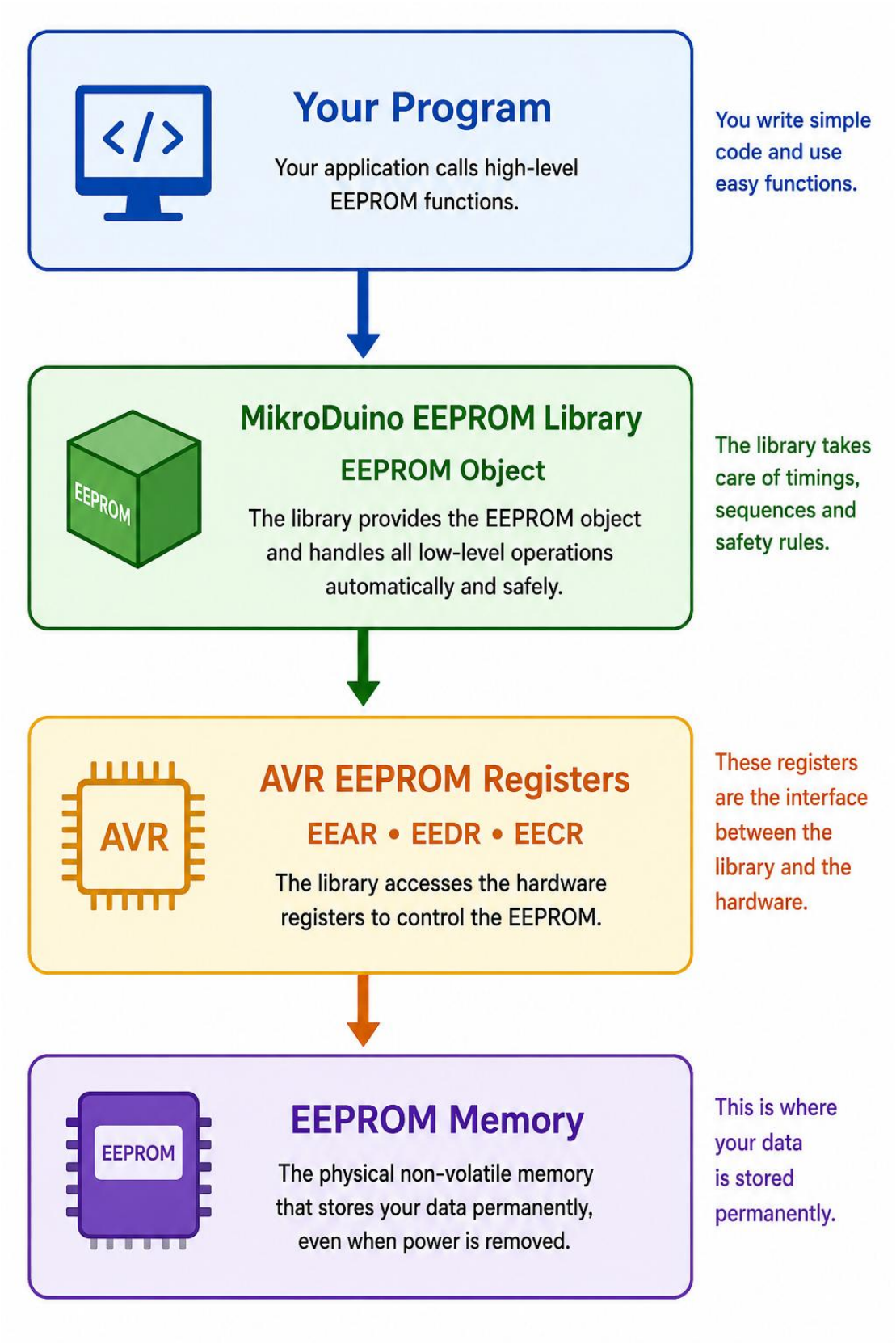
A Layer Above the Hardware

The relationship between your program, the MikroDuino library, and the AVR hardware can be visualized as three layers.



Your application communicates only with the library. The library communicates with the AVR registers. Finally, the registers control the EEPROM hardware.

This layered design makes programs much cleaner while still allowing advanced users to access the registers directly whenever necessary.



Advantages of the MikroDuino EEPROM Library

Using the MikroDuino EEPROM library offers several important advantages over direct register programming.

- Simpler Code: A single function call replaces several register operations.
- Improved Safety: The library automatically follows the correct EEPROM programming sequence, including the EEMPE–EEPE safety mechanism.
- Better Readability: The purpose of the code is immediately obvious.
- Reduced Errors: There is less chance of forgetting a step or violating the four-clock-cycle rule.
- Hardware Independence: The same application code works across supported AVR microcontrollers without modification.

In the following sections, we will explore the functions provided by the MikroDuino EEPROM library. Starting with simple byte-oriented operations such as `read()` and `write()`, we will gradually move on to more advanced features like `update()`, block transfers, and storing complete C++ structures. These high-level functions make permanent data storage remarkably simple while still relying on the same AVR EEPROM hardware and register operations that you now understand.

Reading a Byte

```
uint8_t value = EEPROM.read(100);
```

Writing a Byte

```
EEPROM.write(100,55);
```

Updating a Byte

Updating is slightly optimized write, since write operation itself tends to wear out the memory location, update first checks if the location data is same as we are intending to write, if so there is no need to write.

The `EEPROM.update()` function writes a byte to EEPROM **only if the new value is different from the value already stored**.

Syntax

```
EEPROM.update(address, value);
```

Parameters

Parameter	Description
address	EEPROM memory location where the byte is stored
value	New byte value to be written (0 – 255)

Example: Storing a Configuration Value

Suppose a device stores the brightness level of an LCD backlight at EEPROM address `0`.

```
#include <EEPROM.h>
#define BRIGHTNESS_ADDRESS 0
void setup()
{
    byte brightness = 80;

    EEPROM.update(BRIGHTNESS_ADDRESS, brightness);
}
void loop()
{
}
```

What Happens?

The first time the program runs:

```
EEPROM[0] = 255  (old value)
New value = 80

Different values
        ↓
EEPROM write performed
```

Later, when the program runs again:

```
EEPROM[0] = 80   (stored value)
New value = 80

Values are identical
        ↓
No EEPROM write
```

The EEPROM cell is preserved because no unnecessary write cycle is used.

EEPROM.write() vs EEPROM.update()

Function	Behaviour	EEPROM Wear
`EEPROM.write()`	Always writes the value	Higher
`EEPROM.update()`	Writes only when value changes	Lower

Extending EEPROM Life

Consider a device that saves a sensor reading every second.

Using EEPROM.write()

```
60 seconds × 60 minutes × 24 hours
= 86,400 writes per day
```

At this rate, an EEPROM cell rated for 100,000 cycles could fail in just over a day.

Using `EEPROM.update()`

If the sensor value changes only occasionally, most write operations are avoided, allowing the EEPROM to operate reliably for many years.

Key Point

> `EEPROM.update()` is a safer alternative to `EEPROM.write()` because it prevents unnecessary write operations and extends the operational life of EEPROM memory.

This function should be preferred whenever data is stored repeatedly during normal operation.

Erasing a Byte

```
EEPROM.erase(100);
```

It writes 0xFF in the location. Indicating its empty

Reading and Writing Blocks

Introduction

EEPROM memory is often used to store more than a single byte of information. Many applications need to save **strings, configuration structures, calibration tables, or arrays of data**.

Although EEPROM can be accessed byte-by-byte using `read()` and `write()`, handling larger amounts of data byte-by-byte can make programs longer and more difficult to maintain.

To simplify this process, the EEPROM library provides block operations:

- `readBlock()` — Reads multiple bytes from EEPROM
- `writeBlock()` — Writes multiple bytes to EEPROM
- `updateBlock()` — Writes multiple bytes only when data has changed

These functions are especially useful for storing character arrays.

readBlock()

The `readBlock()` function copies a block of bytes from EEPROM into a variable stored in RAM.

Syntax

```
EEPROM.readBlock(address, buffer, length);
```

Parameters

Parameter	Description
address	Starting EEPROM address
buffer	Memory location where data will be copied
length	Number of bytes to read

writeBlock()

The `writeBlock()` function stores a block of data into EEPROM.

Syntax

```
EEPROM.writeBlock(address, buffer, length);
```

Parameters

Parameter	Description
address	Starting EEPROM address
buffer	Data to store
length	Number of bytes to write

Unlike `updateBlock()`, this function writes every byte regardless of whether the data has changed.

updateBlock()

The `updateBlock()` function is the block equivalent of `EEPROM.update()`. It compares each byte in the EEPROM with the new data and writes only the bytes that have changed.

Syntax

```
EEPROM.updateBlock(address, buffer, length);
```

Advantages

- Reduces EEPROM wear
- Prevents unnecessary write cycles
- Ideal for frequently updated settings

Example: Storing a Character Array

In this example, a device stores a user's name in EEPROM.

```
#include <EEPROM.h>
#define NAME_ADDRESS 0
char name[] = "MikroDuino";
char storedName[20];
void setup()
{
    Serial.begin(9600);

    // Store name in EEPROM
    EEPROM.updateBlock(NAME_ADDRESS, name, sizeof(name));

    // Read name back from EEPROM
    EEPROM.readBlock(NAME_ADDRESS, storedName,
      sizeof(name));

    Serial.print("Stored name: ");
    Serial.println(storedName);
}
void loop()
{
}
```

Practical Applications

Block EEPROM functions are commonly used for:

- Saving device names
- Storing Wi-Fi credentials
- Saving calibration parameters
- User configuration menus
- Storing program settings
- Preserving counters and logs

Storing Structures

Introduction

In embedded systems, EEPROM is frequently used to store **device configuration settings**. These settings usually consist of several related values, such as brightness level, sensor thresholds, calibration constants, and device names.

Instead of storing each variable separately, C++ allows us to group related variables into a **structure (`struct`)** and store the entire structure in EEPROM with a single operation.

This makes EEPROM management simpler, safer, and easier to expand.

Defining a Settings Structure

A structure combines different types of data into one logical unit.

```
struct Settings
{
    uint8_t brightness;
    uint16_t threshold;
    char name[12];
};
```

This structure contains:

Member	Data Type	Purpose
brightness	uint8_t	LED/LCD brightness level
threshold	uint16_t	Sensor threshold value
name	char[12]	Device name

A variable of this type can be created:

```
Settings settings;
```

Saving the Structure

The complete structure can be written to EEPROM using `EEPROM.put()`.

```
EEPROM.put(0, settings);
```

Operation

The EEPROM library automatically:

- Calculates the size of the structure.

- Writes each byte of the structure.
- Stores all members starting from EEPROM address 0.

For example:

```
EEPROM Address

0       brightness
1       threshold (low byte)
2       threshold (high byte)
3       name[0]
4       name[1]
...
14      name[11]
```

The programmer does not need to manage individual addresses.

Loading the Structure

The stored settings can be retrieved using `EEPROM.get()`.

```
settings = EEPROM.get<Settings>(0);
```

The EEPROM contents are copied back into the structure:

```
EEPROM
   |
   |
   ▼
Settings structure in RAM

brightness
threshold
name
```

The individual fields are immediately available:

```
Serial.println(settings.brightness);Serial.println(settin
gs.threshold);Serial.println(settings.name);
```

Complete Example

```
#include <EEPROM.h>
struct Settings
{
    uint8_t brightness;
    uint16_t threshold;
    char name[12];
};
```

```
Settings settings;
void setup()
{
    Serial.begin(9600);

    // Create settings
    settings.brightness = 80;
    settings.threshold = 500;

    strcpy(settings.name, "MikroDuino");

    // Save structure
    EEPROM.put(0, settings);


    // Clear RAM variable
    settings = {0};


    // Load structure
    settings = EEPROM.get<Settings>(0);


    // Display values
    Serial.println(settings.brightness);
    Serial.println(settings.threshold);
    Serial.println(settings.name);
}
void loop()
{
}
```

Why Store a Structure Instead of Individual Fields?

1. Simpler Code

Without a structure, each value needs a separate EEPROM address:

```
EEPROM.write(0, brightness);EEPROM.put(1,
threshold);EEPROM.put(3, name);
```

The programmer must manually calculate addresses and ensure they do not overlap.

With a structure:

```
EEPROM.put(0, settings);
```

Everything is stored automatically.

2. Easier Program Expansion

Suppose later we add:

```
struct Settings
{
    uint8_t brightness;
    uint16_t threshold;
    char name[12];
    uint8_t volume;
};
```

With structure storage, the EEPROM code remains unchanged:

```
EEPROM.put(0, settings);
```

The library automatically stores the additional member.

3. Keeps Related Data Together

A device configuration is a single logical object:

```
Device Settings

  ├──── Brightness
  ├──── Threshold
  └──── Name
```

A structure represents this relationship naturally.

4. Reduces Address Management Errors

When storing fields separately, mistakes can occur:

```
EEPROM.put(0, brightness);EEPROM.put(1, threshold);
```

A `uint16_t` requires two bytes, so the second write may overwrite existing data.

Structures avoid manual byte calculations.

5. Portable and Maintainable

The same structure can be used throughout the program:

```
Settings currentSettings;Settings defaultSettings;
```

The EEPROM storage format follows the program definition.

EEPROM.put() vs EEPROM.write()

Function	Data Stored
`EEPROM.write()`	Single byte
`EEPROM.put()`	Any data type (structures, arrays, numbers)

For example:

```
EEPROM.put(0, settings);
```

can store:

- Integers
- Floating point values
- Arrays
- Structures

Important Note: Structure Size

The EEPROM space required is equal to the size of the structure:

```
sizeof(Settings)
```

For this example:

```
Serial.println(sizeof(Settings));
```

may return a value slightly larger than expected because the compiler may add **padding bytes** for memory alignment.

Key Point

> **Using structures with EEPROM.put() and EEPROM.get() allows complete device configurations to be saved and restored as a single unit. It reduces programming errors, simplifies maintenance, and makes EEPROM-based projects easier to expand.**

This technique is widely used in embedded systems for storing firmware settings, calibration data, and user preferences.

Practical Example: Saving LED Brightness as a Structure

One of the most common uses of EEPROM is storing **user settings** that should remain unchanged even after the microcontroller is switched off.

Suppose you are building an LED dimmer. The user can adjust the LED brightness using a potentiometer or push buttons. Every time the user changes the brightness, you want the microcontroller to remember that value.

Without EEPROM, the brightness would return to its default level every time the power is removed.

By storing the setting in EEPROM, the LED will automatically return to the same brightness when the system is powered on again.

Instead of storing a single variable, it is often better to group related settings into a **structure**. This makes the program easier to expand later.

```
#include <mikroduino/eeprom.hpp>

using namespace MikroDuino;

struct LedSettings

{

    uint8_t brightness;

};

LedSettings settings;

int main()

{

    // Restore saved settings

    EEPROM.get(0, settings);

    // If EEPROM is blank, use a default value

    if (settings.brightness == 0xFF)

        settings.brightness = 128;

    // Set LED brightness

    analogWrite(LED_PIN, settings.brightness);

    // User changes brightness

    settings.brightness = 180;

    // Save new setting

   EEPROM.put(0, settings);

    while (1)
```

```
    {

        // Application code

    }

}
```

Practical Project

Building a Persistent Device Configuration Console

In the previous examples, we learned how to store individual variables and simple structures in EEPROM. In a real embedded system, however, configuration data is often entered by a technician or the user through a serial terminal.

In this project, we will combine two MikroDuino SDK libraries:

- **USART Library** – to communicate with a PC terminal.
- **EEPROM Library** – to permanently store configuration data.

The application allows the user to:

- Enter the device name.
- Set LED brightness.
- Set the temperature alarm threshold.
- Save these settings in EEPROM.
- Retrieve and display the saved settings at any time.

This example demonstrates how multiple MikroDuino libraries can work together to build practical embedded applications.

Defining the Configuration Structure

First, define a structure containing all the settings we wish to store.

```
struct DeviceSettings
{
    char name[20];
    uint8_t brightness;
    uint16_t threshold;
};
```

Create a global instance.

```
DeviceSettings settings;
```

Initializing USART

Include the required libraries.

```
#include <mikroduino/usart.hpp>#include 
<mikroduino/eeprom.hpp>
using namespace MikroDuino;
```

Initialize the serial port.

```
USART.begin(9600);
USART.writeLine("================================");USART.
writeLine(" MikroDuino Configuration 
Demo");USART.writeLine("================================")
;
```

Loading Existing Settings

When the microcontroller starts, retrieve the previously saved configuration.

```
EEPROM.get(0, settings);
```

If EEPROM has never been initialized, assign some default values.

```
if(settings.brightness == 0xFF)
{
    strcpy(settings.name,"MikroDuino");
    settings.brightness = 128;
    settings.threshold = 500;
}
```

Main Menu

Display a simple command menu.

```
Commands

SET     Configure device
SHOW    Display stored settings
SAVE    Save settings
HELP    Display commands
```

Complete Program

```
#include <mikroduino/usart.hpp>#include 
<mikroduino/eeprom.hpp>#include <string.h>
using namespace MikroDuino;
```

```
struct DeviceSettings
{
    char name[20];
    uint8_t brightness;
    uint16_t threshold;
};
DeviceSettings settings;
char command[20];
int main()
{
    USART.begin(9600);

    EEPROM.get(0, settings);

    if(settings.brightness == 0xFF)
    {
        strcpy(settings.name,"MikroDuino");
        settings.brightness = 128;
        settings.threshold = 500;
    }

    USART.writeLine("");
    USART.writeLine("MikroDuino EEPROM Configuration");
    USART.writeLine("--------------------------------");
    USART.writeLine("Commands:");
    USART.writeLine("SET");
    USART.writeLine("SHOW");
    USART.writeLine("SAVE");
    USART.writeLine("");

    while(true)
    {
        USART.write("> ");

        USART.readLine(command,sizeof(command));

        if(strcmp(command,"SET")==0)
        {
            USART.write("Device Name : ");

USART.readLine(settings.name,sizeof(settings.name));

            USART.write("Brightness (0-255): ");
            settings.brightness = USART.readInt();

            USART.write("Threshold : ");
            settings.threshold = USART.readInt();

            USART.writeLine("");
            USART.writeLine("Settings updated.");
        }
```

```
        else if(strcmp(command,"SAVE")==0)
        {
            EEPROM.put(0,settings);

            USART.writeLine("Settings saved to EEPROM.");
        }

        else if(strcmp(command,"SHOW")==0)
        {
            USART.writeLine("");

            USART.write("Device Name : ");
            USART.writeLine(settings.name);

            USART.write("Brightness : ");
            USART.writeInt(settings.brightness);
            USART.writeLine("");

            USART.write("Threshold : ");
            USART.writeInt(settings.threshold);
            USART.writeLine("");

            USART.writeLine("");
        }

        else
        {
            USART.writeLine("Unknown command.");
        }
    }
}
```

Example Terminal Session

After downloading the program, open the serial terminal.

```
MikroDuino EEPROM Configuration
----------------------------
Commands

SET
SHOW
SAVE

>
```

The user enters

```
SET
```

The board asks for the configuration values.

```
Device Name : My Controller

Brightness (0-255): 180

Threshold : 725

Settings updated.
```

Now save them permanently.

```
>SAVE

Settings saved to EEPROM.
```

At any time, display the stored configuration.

```
>SHOW

Device Name : My Controller
Brightness  : 180
Threshold   : 725
```

Now disconnect power from the board.

Reconnect power and type

```
SHOW
```

The terminal displays exactly the same values.

```
Device Name : My Controller
Brightness  : 180
Threshold   : 725
```

This demonstrates that the configuration was successfully stored in EEPROM and survived the power cycle.

How the Program Works

The application follows a simple sequence:

- The microcontroller starts and initializes the USART.
- Previously stored settings are loaded from EEPROM.
- The user interacts with the program through the serial terminal.
- The SET command modifies the settings in RAM.
- The SAVE command stores the entire structure in EEPROM using a single call to EEPROM.put().
- The SHOW command displays the current configuration by reading the values already loaded into RAM.
- After a power cycle, EEPROM.get() restores the configuration, allowing the device to continue operating with the same settings as before.

This example highlights one of the greatest strengths of the MikroDuino SDK: multiple libraries can be combined seamlessly. The **USART library** provides an intuitive interface for communicating with the user, while the **EEPROM library** handles all of the low-level register operations required for non-volatile storage. By representing all configuration parameters as a single C++ structure, the entire device configuration can be saved and restored with just two simple function calls, resulting in code that is both easy to understand and easy to maintain.

Best Practices for Using EEPROM

EEPROM is one of the most valuable resources available in an AVR microcontroller. It allows important information to be preserved even when the power is removed. However, unlike SRAM, EEPROM has a limited number of write cycles. Careless programming can shorten its lifetime or make configuration data unreliable.

By following a few simple guidelines, you can ensure that your EEPROM-based applications remain reliable for many years.

Never Write Continuously Inside `while(1)`

One of the most common mistakes made by beginners is writing data to EEPROM inside an infinite loop.

For example,

```
while (1)
{
    EEPROM.write(0, sensorValue);
}
```

At first glance, this may seem harmless, but it is actually a serious programming error.

Remember that EEPROM can typically withstand about **100,000 write cycles** per memory location. If the program writes continuously inside a fast loop, this limit can be reached within a short time, permanently reducing the reliability of that EEPROM location.

A much better approach is to write data **only when it actually changes** or when the user explicitly requests it.

For example, save settings only when the **Save** button is pressed or when the user exits a configuration menu.

Prefer `update()` Instead of `write()`

The `write()` function always programs the EEPROM, even if the new value is exactly the same as the value already stored.

For example,

```
EEPROM.write(10, 100);
```

will perform a write operation every time it is executed.

The `update()` function is smarter.

```
EEPROM.update(10, 100);
```

Before writing, it first reads the existing value.

- If the value is different, it performs the write.
- If the value is already 100, no write occurs.

This simple optimization greatly reduces unnecessary write cycles and significantly increases EEPROM life.

Whenever possible, prefer `update()` over `write()`.

Reserve Fixed EEPROM Addresses

As projects become larger, multiple pieces of information must be stored.

For example,

- Device name
- LED brightness
- Wi-Fi password
- Calibration data
- Operating mode

If these values are written to random addresses, the program quickly becomes confusing.

Instead, assign fixed EEPROM locations to each parameter.

For example,

EEPROM Address	Purpose
0	Device Settings
32	Wi-Fi Configuration
96	Calibration Data
160	Error Log
224	Reserved for Future Use

Having a predefined layout makes the program easier to understand and maintain.

Use Structures for Organized Storage

Instead of storing each variable individually, group related information into a structure.

Instead of

```
EEPROM.write(0, brightness);EEPROM.write(1, mode);EEPROM.write(2, threshold);
```

define a structure.

```
struct Settings
{
    uint8_t brightness;
    uint8_t mode;
    uint16_t threshold;
};
```

Now the complete configuration can be saved using

```
EEPROM.put(0, settings);
```

and restored using

```
EEPROM.get(0, settings);
```

This approach produces cleaner, shorter, and more maintainable programs.

It also makes future expansion much easier because new variables can simply be added to the structure.

Check for Factory-Erased EEPROM (`0xFF`)

A brand-new AVR microcontroller usually contains **0xFF** in every EEPROM location.

Similarly, after an EEPROM erase operation, every byte becomes

```
11111111
```

or

```
0xFF
```

If your application immediately reads configuration data without checking for this condition, it may interpret **0xFF** as valid information.

Instead, verify whether EEPROM has been initialized.

For example,

```
EEPROM.get(0, settings);
if (settings.brightness == 0xFF)
{
    settings.brightness = 128;
}
```

In this example, the program detects an uninitialized EEPROM and loads sensible default values instead.

Store a Magic Number

Checking a single variable for **0xFF** is useful, but it is not always reliable. A better technique is to store a **magic number** at the beginning of the EEPROM.

A magic number is simply a known value that identifies valid configuration data.

For example,

```
struct Settings
{
    uint16_t magic;
    uint8_t brightness;
    uint16_t threshold;
};
```

When saving the configuration,

```
settings.magic = 0x55AA;
EEPROM.put(0, settings);
```

During startup,

```
EEPROM.get(0, settings);
if (settings.magic != 0x55AA)
{
    // EEPROM not initialized
}
```

If the magic number is missing or incorrect, the program knows that the EEPROM contents are invalid or belong to an older firmware version. It can then safely load default settings instead of using corrupted or uninitialized data.

Keep an EEPROM Memory Map

For very small programs, remembering EEPROM addresses is easy. As projects become larger, however, dozens of variables may be stored in EEPROM.

Without proper documentation, it becomes difficult to remember which address stores what information.

For this reason, maintain a simple EEPROM memory map.

For example,

Address Range	Contents	Size
0 – 31	Device Settings	32 Bytes
32 – 95	User Profile	64 Bytes
96 – 127	Calibration Data	32 Bytes
128 – 191	Event Log	64 Bytes
192 – 255	Reserved	64 Bytes

This table serves as a blueprint for your EEPROM layout and helps prevent different parts of the program from accidentally overwriting each other's data.

Minimize Write Operations

Another useful guideline is to write data **only when necessary**.

For example, a temperature sensor that updates every second should not store every reading in EEPROM. Instead, store only important configuration values or data that must survive a power failure.

Frequent measurements belong in **SRAM**, while long-term configuration belongs in **EEPROM**.

Use Meaningful Constants for EEPROM Addresses

Avoid scattering numerical addresses throughout your program.

Instead of writing

```
EEPROM.put(32, settings);
```

define a named constant.

```
const uint16_t SETTINGS_ADDRESS = 32;
EEPROM.put(SETTINGS_ADDRESS, settings);
```

This makes the program much easier to understand and allows EEPROM addresses to be changed in one place if the memory layout is updated later.

Summary

EEPROM is designed to store valuable information that must survive power loss, such as user settings, calibration values, and device configuration. To make the most

of this limited but extremely useful resource, write to EEPROM only when necessary, prefer `update()` over `write()`, organize related data using structures, and maintain a well-planned memory layout. Checking for uninitialized memory and validating stored data with a magic number further improves reliability. By following these best practices, your EEPROM-based applications will remain robust, easier to maintain, and capable of preserving important information throughout the lifetime of the microcontroller.